

**GOBLIN**

Grin wallet · nostr · Nym mixnet

Payment Transport Overview

How a slatepack travels end to end — wrapped, mixed, and finalized.

Goblin leaves Grin's money path untouched and replaces exactly one thing: how the slatepack gets from sender to recipient. Instead of exporting a file and shipping it out of band, the wallet wraps each slatepack as an end-to-end-encrypted nostr message and routes it over the Nym mixnet — addressed to a username rather than a copy-pasted blob.

It covers what moves on the wire: the round trip step by step, the mixnet-versus-direct split, the nostr kinds and NIPs that carry it, and the security model. Every NIP below links to its spec.

ENCRYPTION · IN TRANSPORT & AT REST

In transport

NIP-44 **Payload** — ChaCha20 + HMAC-SHA256; keys via secp256k1 ECDH → HKDF-SHA256

NIP-59 **Seal & gift wrap** — binds the real author, hides sender and recipient from relays

NIP-17 **DM envelope** — the kind-14 message the slatepack rides inside

Sphinx **Network layer** — 5-hop onion routing across the Nym mixnet

At rest

NIP-49 **Key storage** — scrypt + XChaCha20-Poly1305, written to an owner-only file (0600)

Backup **Portable export** — NIP-49 key plus a NIP-44-sealed identity; no npub or name in the clear

CONTENTS

- 1 · Overview — the transport layer in one view
- 2 · The payment flow — slatepack to settlement
- 3 · What rides the mixnet, and what is direct
- 4 · Technical building blocks (nostr kinds & NIPs)
- 5 · Security model

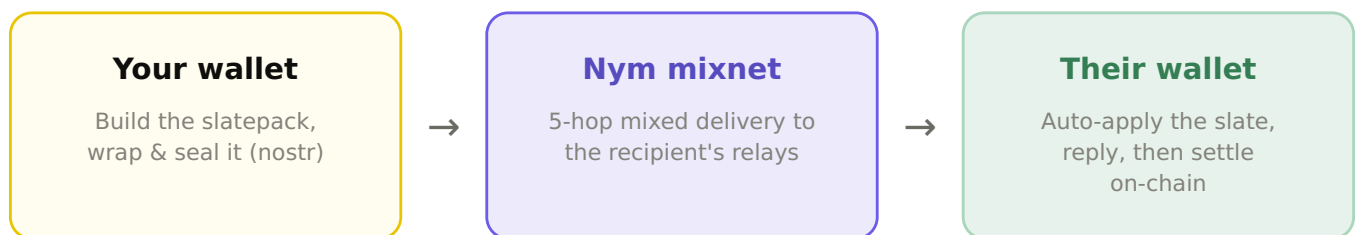


1 · Overview

Grin payments are interactive: a slatepack is built, handed to the counterparty, answered, and finalized. Goblin changes none of that arithmetic. What it replaces is the hand-off — the out-of-band step where a slatepack is exported, shipped over some channel, and imported on the other side. That step is where coordination, metadata leakage, and friction actually live, and it is the only thing this document covers.

Goblin is a fork of the Grim wallet and inherits its transaction logic verbatim; the delivery path is the new code. The wallet wraps each slatepack as an encrypted nostr direct message and routes it over the Nym mixnet to the recipient's published relays. The receiving wallet applies the slate and replies on the same path, with no manual step on either side. Relays buffer the message, so the two wallets need not be online at the same instant.

AT A GLANCE



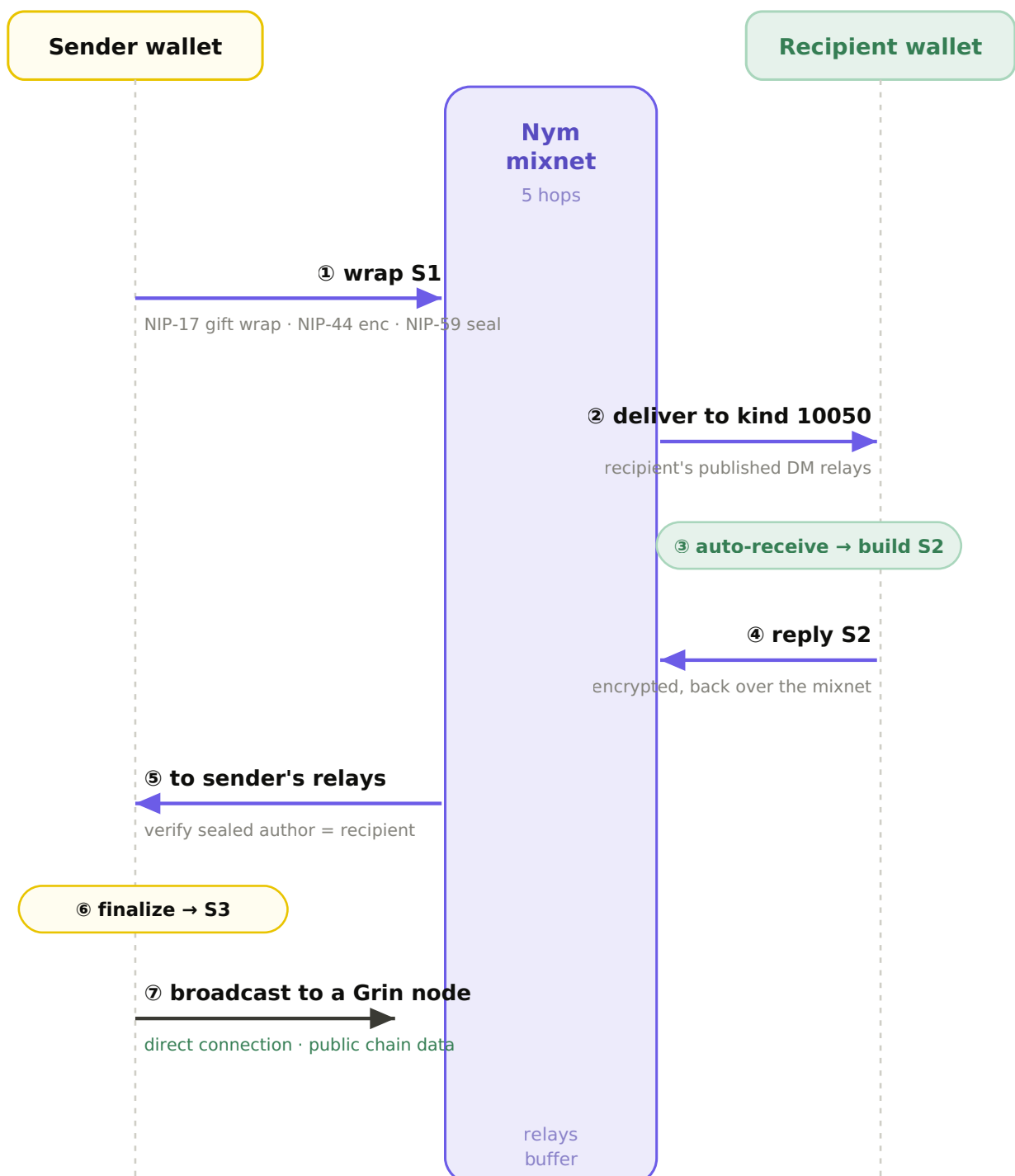
WHY A MIXNET AND NOT JUST ENCRYPTION

Encryption hides a message's contents, not the fact that two parties exchanged one. The slatepack exchange is a recognizable cadence — S1 out, S2 back, finalize — so an observer at the network edges can correlate the legs by timing even when every byte is encrypted. That timing linkage is precisely the metadata the rest of the stack works to remove.

A mixnet breaks the linkage. Nym batches traffic and adds independent per-hop timing jitter across five Sphinx hops, so S1 and its reply no longer line up on a stopwatch. The entry gateway also buffers traffic for offline clients — the same mailbox property the relays already provide.

2 · The payment flow

A send and its automatic reply, step by step. S1 is the sender's partial, S2 the recipient's reply; the sender finalizes to S3 and broadcasts. A payment request uses the mirror states I1/I2 and is never auto-paid — the payer always approves.



3 · What rides the mixnet

Goblin is deliberate about its privacy claims: over-claiming is worse than being clear. Two lanes leave your wallet.

Over the Nym mixnet · 5 hops

- Payment messages — nostr gift wraps (the slatepack itself)
- Username lookups — NIP-05 name resolution
- Price feed · contact avatars

Everything tied to you and to who you pay is mixed.

Direct connection

- Grin node — block sync and broadcasting your transaction
- Public chain data — the same blocks everyone downloads
- Not tied to your identity · point it at any public node, or run your own

This is how Grim works; Goblin does not change the money path.

Stated in the wallet

The split is shown under Settings → Mixnet routing → "Messages & lookups", with a one-tap breakdown of each channel — so the guarantee is legible, not buried.

Goblin ships a redundant list of community Grin nodes (mainnet and testnet) so a single operator going down never strands the wallet, and new wallets default to an instant public-node connection rather than waiting on a full local sync.

4 · Building blocks

WHAT EACH PIECE DOES

<u>NIP-17</u>	Private direct messages	the slatepack travels as a kind-14 chat message
<u>kind 1059</u>	Gift wrap	outer wrapper that hides sender, recipient and content
<u>NIP-59</u>	Seal	inner seal binds the real author; a forged sender is rejected
<u>NIP-44</u>	Encryption	versioned, authenticated payload encryption
<u>kind 10050</u>	DM relay list	where to drop a message so the recipient will find it
<u>kind 0</u>	Profile metadata	confirms a key is a live identity before you pay it
<u>NIP-05</u>	Name → key	resolves username@domain to a pubkey (the goblin.st service)
<u>NIP-98</u>	HTTP auth	signed-event auth to claim or move a name, no passwords

IDENTITY: NOT YOUR MONEY

Your nostr payment key is generated independently of your Grin wallet seed. It can be rotated at any time to stay unlinkable, and a future seed compromise can never reach it. The optional human-readable name lives in an open NIP-05 name service the community can self-host for redundancy — the key always wins; the name is a convenience layer on top.

SLATE STATES

S1 → S2 → S3

a send: sender's partial, recipient's reply, finalize & broadcast

I1 → I2

a request (invoice): never auto-paid — the payer approves



5 · Security model

Sealed authorship

The NIP-59 seal is verified on receipt: the message's real author must equal the sealed sender, and the inner rumor must be unsigned. A wrap whose author was forged is dropped.

Counterparty binding

An incoming reply or finalize must match the stored counterparty, direction, and state of an existing transaction. A stranger's slate for your in-flight payment is rejected, not applied.

Payments final, requests are messages

Incoming payments auto-apply; incoming requests never do. Declining or cancelling a request notifies the other side, but no money moves without explicit approval.

Encryption everywhere

Message contents use NIP-44 authenticated encryption inside the gift wrap; the transport adds the mixnet on top. Relays and mix nodes see neither contents nor the link between sender and recipient.

Identity at rest

The secret key is stored encrypted (NIP-49, scrypt) with owner-only file permissions and is never written to disk or logs in plaintext — only exported on an explicit user backup.

Resilient receiving

A momentary wallet or node hiccup while receiving does not drop the payment: it is retried on the next sync rather than silently lost, and finalizing is idempotent.

Honest about what it is

Goblin is beta. The cryptography is Grin's and nostr's; the new part is the plumbing. Start with small amounts, back up your seed, read the code.